



Windows Endpoint SOC Simulation using ELK & Sysmon

By Chontele Coleman

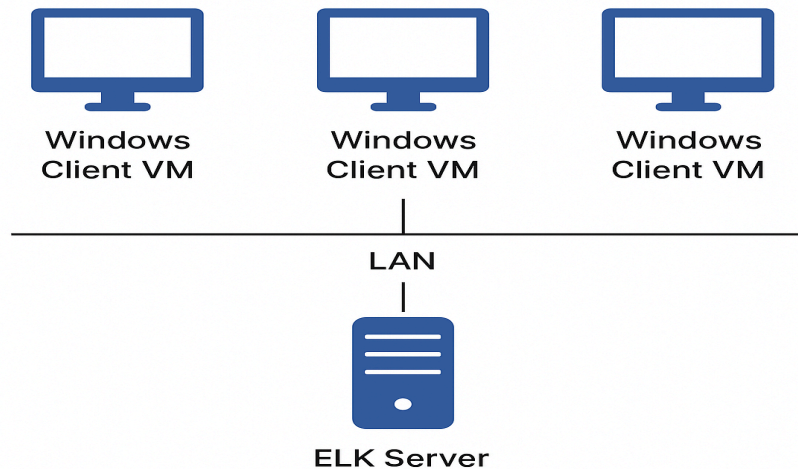
November 12, 2025

Overview

This project simulates a Security Operations Center (SOC) environment to monitor Windows endpoints using Sysmon, Winlogbeat, and the ELK Stack. My goal is to capture, visualize, and analyze various Windows event logs to demonstrate capabilities in incident detection, PowerShell monitoring, registry and file activity, Windows Defender events, and lateral movement.

Lab Environment

- Windows 10 VM: Endpoint monitoring agent and Sysmon installed.
- Windows Server 2019 VM: Used to simulate attacker activities.
- Ubuntu VM: Hosts Elasticsearch, Logstash, and Kibana (ELK Stack) for log aggregation and visualization.

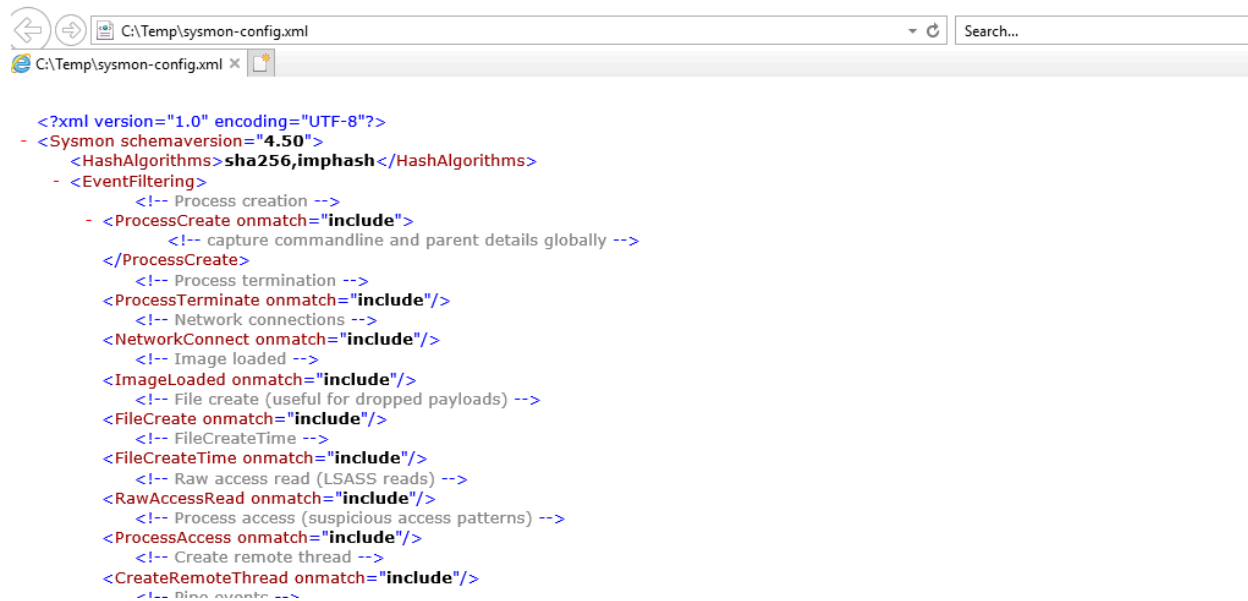


Caption: Lab environment showing Windows endpoints and ELK server connections.

Configuration Summary

Sysmon Configuration:

- Installed on both Windows 10 and Windows Server 2019.
- Monitors process creation, registry changes, file creation, network connections, and more.

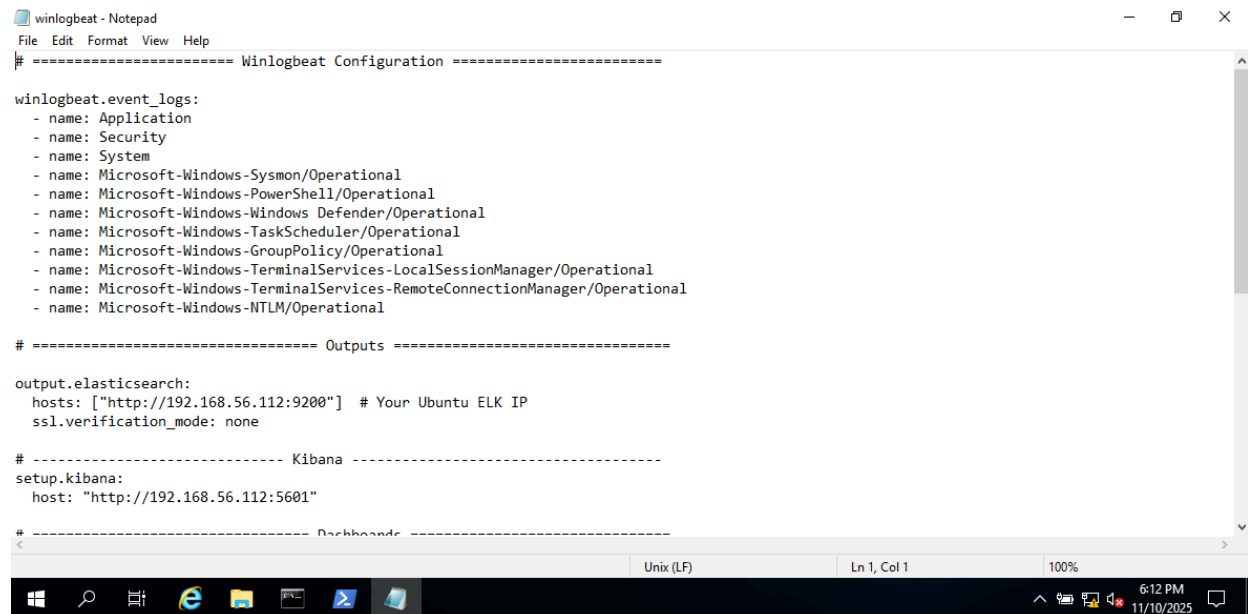
A screenshot of a text editor window titled 'C:\Temp\sysmon-config.xml'. The editor shows an XML configuration for Sysmon. The root element is <?xml version="1.0" encoding="UTF-8"?>. Below it is <Sysmon schemaversion="4.50">. Inside this, there's <HashAlgorithms>sha256,imphash</HashAlgorithms>. Then <EventFiltering> with several <!-- Process creation --> and <!-- Process termination --> comments. The <ProcessCreate onmatch="include"/> element is followed by a <!-- capture commandline and parent details globally --> comment. Then </ProcessCreate>, <!-- Process termination -->, <ProcessTerminate onmatch="include"/>, <!-- Network connections -->, <NetworkConnect onmatch="include"/>, <!-- Image loaded -->, <ImageLoaded onmatch="include"/>, <!-- File create (useful for dropped payloads) -->, <FileCreate onmatch="include"/>, <!-- FileCreateTime -->, <FileCreateTime onmatch="include"/>, <!-- Raw access read (LSASS reads) -->, <RawAccessRead onmatch="include"/>, <!-- Process access (suspicious access patterns) -->, <ProcessAccess onmatch="include"/>, <!-- Create remote thread -->, <CreateRemoteThread onmatch="include"/>, and finally <!-- Misc events -->.

```
<?xml version="1.0" encoding="UTF-8"?>
- <Sysmon schemaversion="4.50">
  <HashAlgorithms>sha256,imphash</HashAlgorithms>
  - <EventFiltering>
    <!-- Process creation -->
    - <ProcessCreate onmatch="include">
      <!-- capture commandline and parent details globally -->
    </ProcessCreate>
    <!-- Process termination -->
    <ProcessTerminate onmatch="include"/>
    <!-- Network connections -->
    <NetworkConnect onmatch="include"/>
    <!-- Image loaded -->
    <ImageLoaded onmatch="include"/>
    <!-- File create (useful for dropped payloads) -->
    <FileCreate onmatch="include"/>
    <!-- FileCreateTime -->
    <FileCreateTime onmatch="include"/>
    <!-- Raw access read (LSASS reads) -->
    <RawAccessRead onmatch="include"/>
    <!-- Process access (suspicious access patterns) -->
    <ProcessAccess onmatch="include"/>
    <!-- Create remote thread -->
    <CreateRemoteThread onmatch="include"/>
    <!-- Misc events -->
  </EventFiltering>
</Sysmon>
```

Caption: Sysmon configuration applied to monitor key events.

Winlogbeat Configuration:

- Configured to ship logs from multiple Windows event channels to ELK.
- Captures Sysmon, PowerShell, Defender, Task Scheduler, and other operational logs.

A screenshot of a Notepad window titled 'winlogbeat - Notepad'. The window shows the Winlogbeat configuration file. The configuration includes event_logs, outputs, and setup.kibana sections. The event_logs section lists several Windows event channels. The outputs section shows the Elasticsearch host and SSL verification mode. The setup.kibana section shows the Kibana host.

```
winlogbeat.event_logs:
- name: Application
- name: Security
- name: System
- name: Microsoft-Windows-Sysmon/Operational
- name: Microsoft-Windows-PowerShell/Operational
- name: Microsoft-Windows-Windows Defender/Operational
- name: Microsoft-Windows-TaskScheduler/Operational
- name: Microsoft-Windows-GroupPolicy/Operational
- name: Microsoft-Windows-TerminalServices-LocalSessionManager/Operational
- name: Microsoft-Windows-TerminalServices-RemoteConnectionManager/Operational
- name: Microsoft-Windows-NTLM/Operational

# ===== Outputs =====

output.elasticsearch:
  hosts: ["http://192.168.56.112:9200"] # Your Ubuntu ELK IP
  ssl.verification_mode: none

# ----- Kibana -----
setup.kibana:
  host: "http://192.168.56.112:5601"
```

Caption: Winlogbeat configuration for shipping Windows logs to ELK.

Example Verification Commands:

sysmon.exe -c C:\Temp\sysmon-config.xml

```
PS C:\Users\Administrator> Get-Service -Name Sysmon64 | Format-List Name, Status, DisplayName
Name       : Sysmon64
Status     : Running
DisplayName : Sysmon64

PS C:\Users\Administrator> Test-Path "$env:windir\System32\drivers\sysmon.sys"
False
PS C:\Users\Administrator> Get-ChildItem "$env:windir\System32\drivers\sysmon*" | Format-List FullName, Length, LastWriteTime
PS C:\Users\Administrator> C:\Temp\Sysmon64.exe -c

System Monitor v15.15 - System activity monitor
By Mark Russinovich and Thomas Garnier
Copyright (C) 2014-2024 Microsoft Corporation
Using libxml2. libxml2 is Copyright (C) 1998-2012 Daniel Veillard. All Rights Reserved.
Sysinternals - www.sysinternals.com

Current configuration:
- Service name:           Sysmon64
- Driver name:            SysmonDrv
- Config file:            "C:\Temp\Sysmon64.exe" -accepteula -i
- HashingAlgorithms:      SHA256
- Network connection:     disabled
- Archive Directory:       -
- Image loading:           disabled
- CRL checking:            enabled
- DNS lookup:             enabled

No rules installed
PS C:\Users\Administrator> Get-WinEvent -ListLog "Microsoft-Windows-Sysmon/Operational"

LogMode  MaximumSizeInBytes RecordCount LogName
-----
Circular  67108864      29 Microsoft-Windows-Sysmon/Operational
```

Get-Service Winlogbeat

```
PS C:\Users\Administrator> Start-Service -Name winlogbeat
PS C:\Users\Administrator> Get-Service -Name winlogbeat

Status  Name      DisplayName
-----
Running winlogbeat winlogbeat

PS C:\Users\Administrator> .
```

Dashboards & Visualizations

All dashboards are built in Kibana using queries against the ingested Windows logs. Each visualization captures a specific area of endpoint monitoring.

Dashboard 1: System Activity Monitoring

Purpose: Track creation of new files for anomaly detection.

```
Administrator: Windows PowerShell
PS C:\Users\Administrator> notepad.exe
PS C:\Users\Administrator> calc.exe
PS C:\Users\Administrator> cmd.exe /c "whoami && ipconfig"
corp\administrator

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::8738:e941:36f6:3fa5%6
    IPv4 Address. . . . . : 192.168.56.108
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix  . : lan
    IPv6 Address. . . . . : fd17:625c:f037:3:bb06:8721:c6c2:7b6
    Temporary IPv6 Address. . . . . : fd17:625c:f037:3:4d09:7933:5179:76d
    Link-local IPv6 Address . . . . . : fe80::40c0:165d:216f:e94c%12
    IPv4 Address. . . . . : 10.0.3.15
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : fe80::2%12
                               10.0.3.2

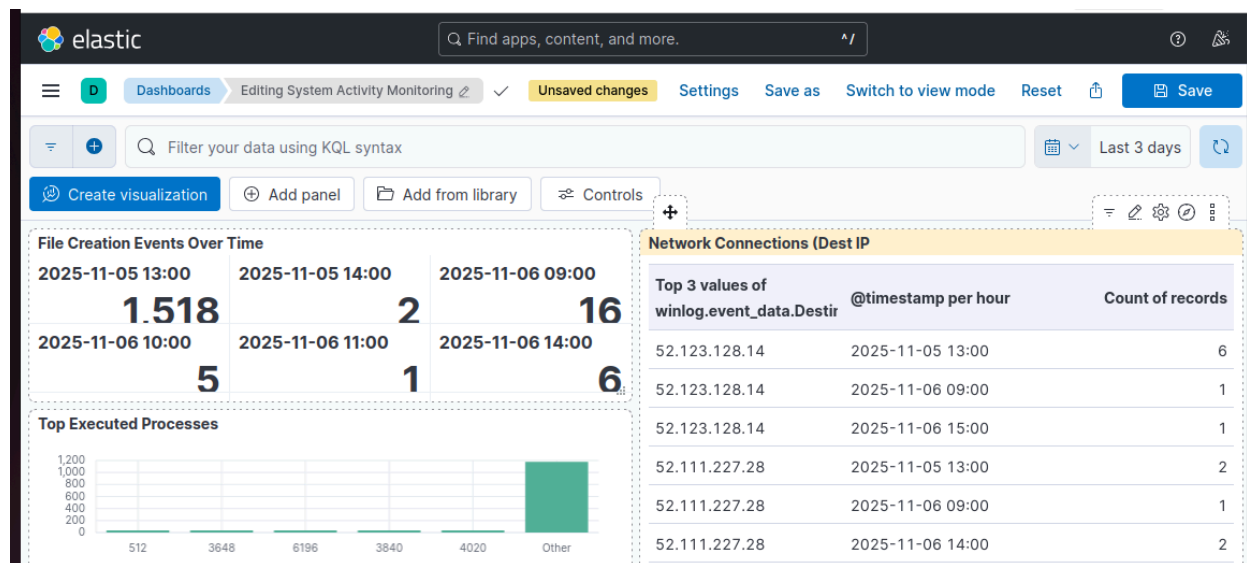
PS C:\Users\Administrator> curl.exe https://example.com
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="viewport" content="width=device-width, ini
tial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5e
m}div{opacity:0.8}a:link,a:visited{color:#348}</style><body><div><h1>Example Domain</h1><p>This domain is for use in doc
umentation examples without needing permission. Avoid use in operations.<p><a href="https://iana.org/domains/example">Le
arn more</a></div></body></html>

PS C:\Users\Administrator> ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=33ms TTL=255
Reply from 8.8.8.8: bytes=32 time=30ms TTL=255
Reply from 8.8.8.8: bytes=32 time=35ms TTL=255
Reply from 8.8.8.8: bytes=32 time=28ms TTL=255

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 28ms, Maximum = 35ms, Average = 31ms

PS C:\Users\Administrator> echo "sysmon test log" > C:\Temp\sysmon_test.txt
PS C:\Users\Administrator> copy C:\Temp\sysmon_test.txt C:\Temp\sysmon_test_copy.txt
PS C:\Users\Administrator>
```



Dashboard 2: Authentication Logon Events

Purpose: centralizes visibility into user login activity.

```
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64> # Successful logon
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64> runas /user:Administrator cmd.exe
Enter the password for Administrator:
Attempting to start cmd.exe as user "CORP\Administrator" ...
RUNAS ERROR: Unable to run - cmd.exe
1326: The user name or password is incorrect.

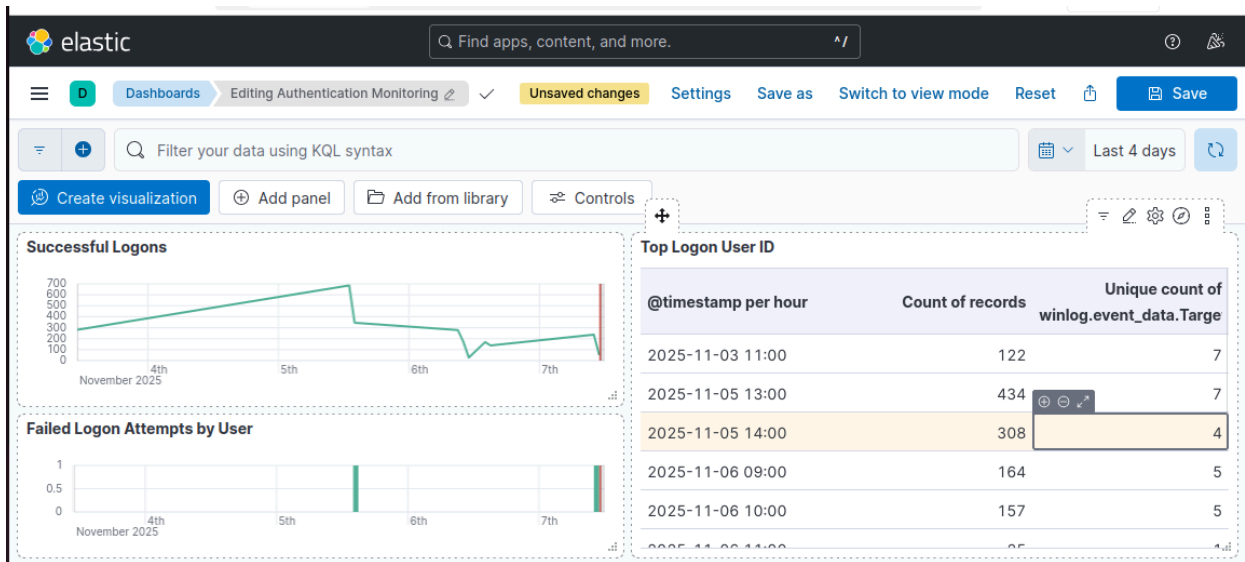
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64>
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64> # Failed logons (wrong password)
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64> for ($i=0; $i -lt 3; $i++) { runas /user:Administrator cmd.exe /savecred }
RUNAS USAGE:

RUNAS [ [/noprofile | /profile] [/env] [/savecred | /netonly] ]
      /user:<UserName> program

RUNAS [ [/noprofile | /profile] [/env] [/savecred] ]
      /smartcard [/user:<UserName>] program

RUNAS /trustlevel:<TrustLevel> program

      /noprofile      specifies that the user's profile should not be loaded.
                     This causes the application to load more quickly, but
                     can cause some applications to malfunction.
      /profile        specifies that the user's profile should be loaded.
                     This is the default.
      /env            to use current environment instead of user's.
      /netonly        use if the credentials specified are for remote
                     access only.
      /savecred       to use credentials previously saved by the user.
      /smartcard      use if the credentials are to be supplied from a
                     smartcard.
      /user           <UserName> should be in form USER@DOMAIN or DOMAIN\USER
      /showtrustlevels displays the trust levels that can be used as arguments
                     to /trustlevel.
      /trustlevel     <Level> should be one of levels enumerated
```

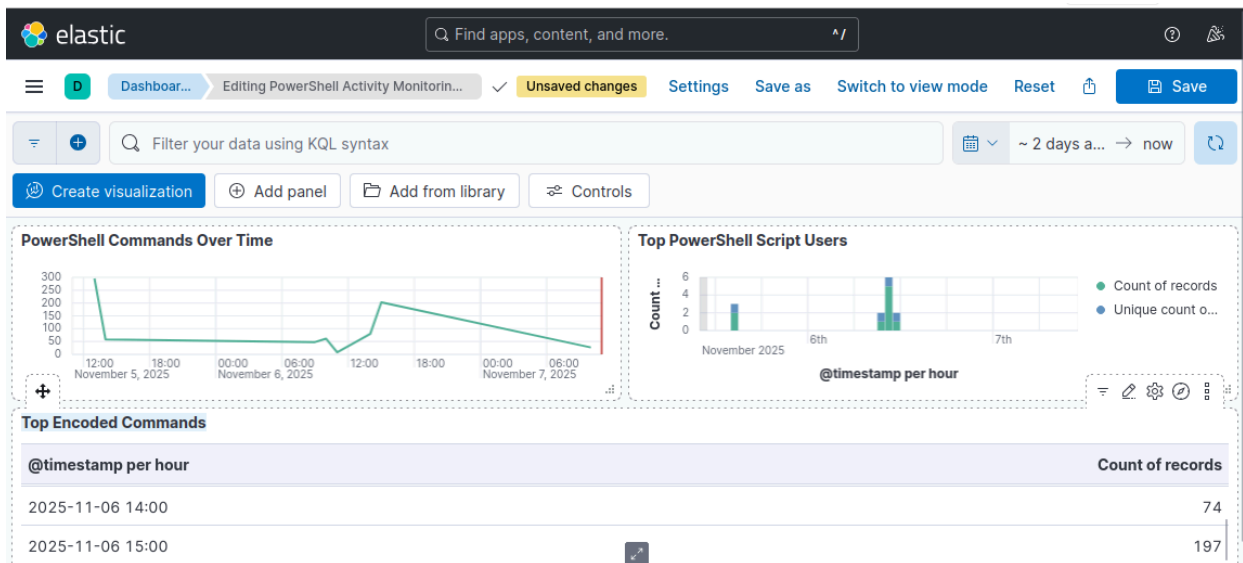


Dashboard 3: PowerShell Activity

Purpose: Capture PowerShell executions (Event ID 4104).

```
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64> # Benign PowerShell activity
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64> Get-Process
```

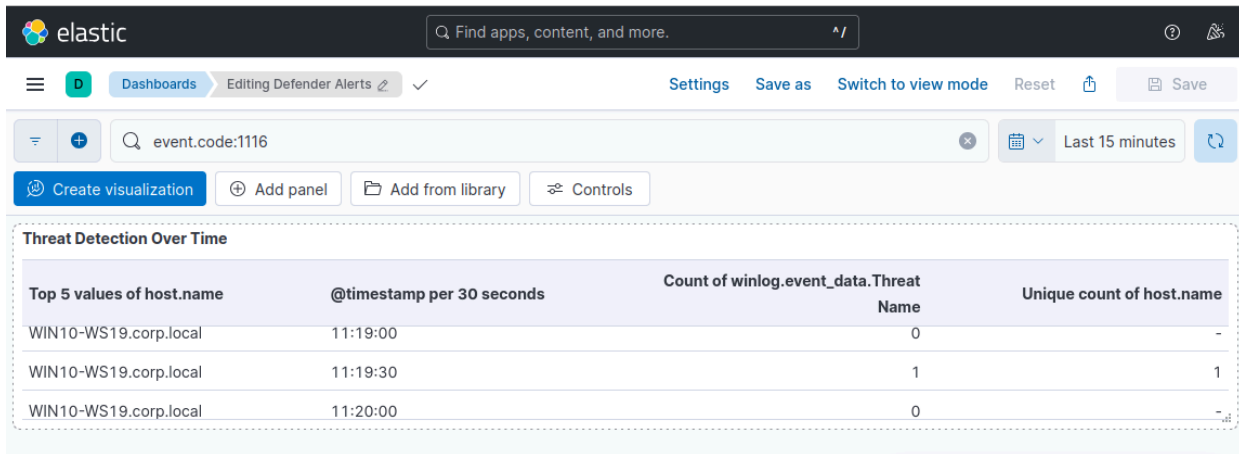
Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
257	13	4288	17696	6.30	1148	2	conhost
478	19	2328	5952	0.50	380	0	csrss
279	14	2004	5204	10.92	3484	2	csrss
372	15	3528	15032	0.31	5516	2	ctfmon
395	32	15580	21480	1.63	592	0	dfsrs
158	8	1924	6104	0.00	3356	0	dfssvc
194	16	3084	10512	0.08	4024	0	dllhost
10386	7311	130680	129204	3.50	3108	0	dns
591	35	24008	50748	12.98	5852	2	dwm
1429	55	21328	50276	2.78	5900	2	explorer
53	6	1420	4372	0.03	2908	0	fontdrvhost
53	7	1868	5616	0.22	5860	2	fontdrvhost
0	0	56	8	0	0	0	Idle
134	12	1852	5672	0.06	3132	0	ismserv
1815	185	59052	70112	11.38	604	0	lsass
433	30	31248	42124	1.25	2796	0	Microsoft.ActiveDirectory.WebServices
492	17	9368	21624	0.72	3148	0	MpDefenderCoreService
232	13	3208	10448	0.28	5592	0	msdtc
564	217	255648	58584	152.78	3264	0	MsMpEng
694	30	61840	105312	3.11	4364	2	powershell
0	7	344	61688	0.70	88	0	Registry
307	16	5508	17276	0.25	464	2	RuntimeBroker
233	12	2668	16940	0.05	2184	2	RuntimeBroker
402	21	6288	22088	0.33	6012	2	RuntimeBroker
667	32	19444	61700	0.77	3772	2	SearchUI
648	14	5764	13472	3.69	596	0	services
865	32	21396	67468	1.19	3100	2	ShellExperienceHost
471	17	4944	24852	0.53	3780	2	sihost
53	3	484	1140	0.16	288	0	smss
477	22	5720	17080	0.16	2728	0	spoolsv
271	13	3568	11272	0.34	332	0	svchost



Dashboard 4: Windows Defender & Security Alerts

Purpose: Surface malware detections or Defender events.

```
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64> # Generate harmless Defender log
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64> Get-MpComputerStatus | Out-File C:\Temp\defender_status.txt
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64>
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64> # Trigger test alert (safe EICAR test file)
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64> echo "X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*" > C:\Temp\eicar.com
PS C:\Program Files\winlogbeat-9.1.1-windows-x86_64>
```



Dashboard 5: Lateral Movement & Remote Activity

Purpose: Monitor remote service creation, WMI execution, and remote thread creation.

```

Circular      20971520      17133 System

PS C:\Users\Administrator> # Create service on the remote Win10 target
PS C:\Users\Administrator> sc.exe \\192.168.56.108 create TestSvc binPath= "cmd.exe /c whoami" start= demand
[SC] CreateService SUCCESS
PS C:\Users\Administrator>
PS C:\Users\Administrator> # Start the service (this runs the cmd on target and generates events)
PS C:\Users\Administrator> sc.exe \\192.168.56.108 start TestSvc
[SC] StartService FAILED 1053:

The service did not respond to the start or control request in a timely fashion.

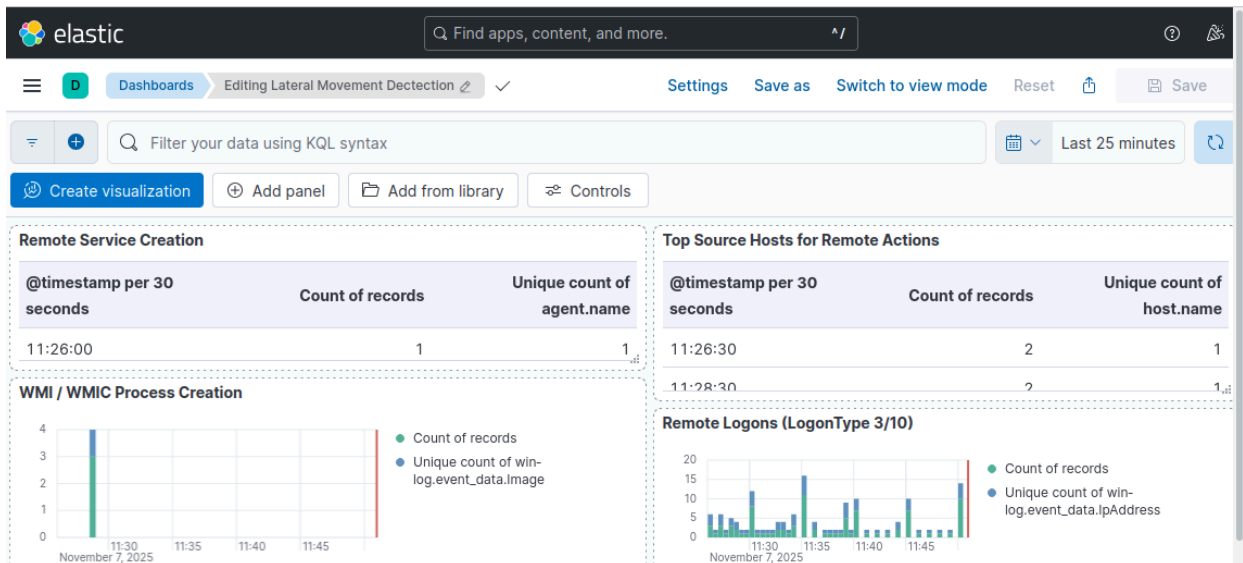
PS C:\Users\Administrator>
PS C:\Users\Administrator> Start-Sleep -Seconds 3
PS C:\Users\Administrator>
PS C:\Users\Administrator> # Stop & delete
PS C:\Users\Administrator> sc.exe \\192.168.56.108 stop TestSvc
[SC] ControlService FAILED 1062:

The service has not been started.

PS C:\Users\Administrator> sc.exe \\192.168.56.108 delete TestSvc
[SC] DeleteService SUCCESS
PS C:\Users\Administrator> # Create a process remotely using WMIC (requires admin on target)
PS C:\Users\Administrator> wmic /node:192.168.56.108 process call create "cmd.exe /c whoami > C:\Temp\whoami.txt"
Executing (Win32_Process)->Create()
Method execution successful.
Out Parameters:
instance of __PARAMETERS
{
    ProcessId = 1520;
    ReturnValue = 0;
};

PS C:\Users\Administrator> .\PsExec.exe \\192.168.56.108 -accepteula -s cmd.exe /c whoami

```



Simulated Alert Data

The following index was created to hold simulated alerts:

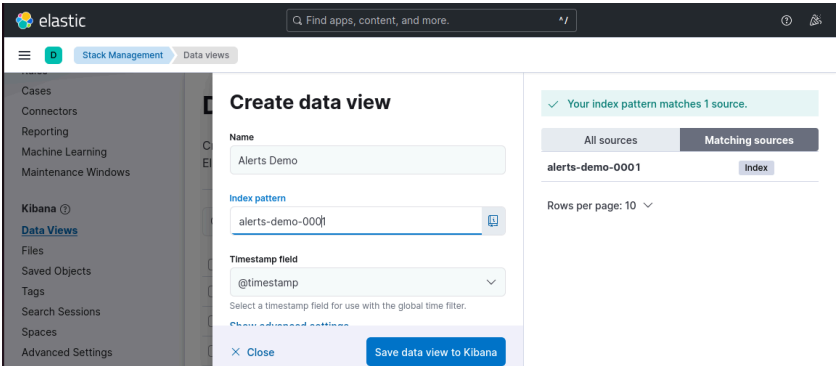
curl -X PUT "http://192.168.56.112:9200/alerts-demo-0001"

A sample JSON file (alerts-demo.json) or CSV file can be uploaded to this index containing fake alerts such as:

@timestamp	rule.name	host.name	event.action	severity	message
2025-11-10T10:00:00Z	Failed Logon Attempt	workstation01	login_failed	High	User failed multiple logins
2025-11-10T10:05:00Z	Firewall Block Event	ubuntu-srv01	network_block	Medium	Suspicious IP connection blocked

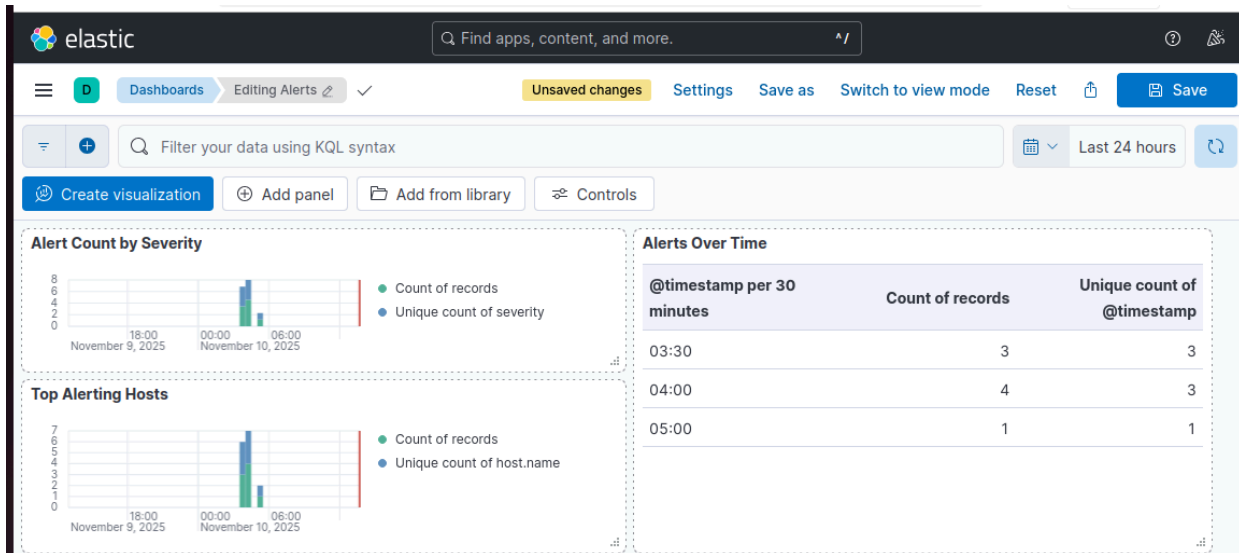
```
colenan@ubuntu:~$ curl -X POST "http://192.168.56.112:9200/alerts-demo-0001/_bulk" \
-H "Content-Type: application/x-ndjson" \
--data-binary "@alerts-demo.ndjson"
{"errors":false,"took":0,"items":[{"_index":{"_index":"alerts-demo-0001","_id":"3xT1bp0BfNVXN1ie2VG","_version":1,"result":"created","_shards":{"total":2,"successful":1,"failed":0},"_seq_no":1,"_primary_term":1,"status":201}},{"_index":{"_index":"alerts-demo-0001","_id":"48T1bp0BfNVXN1ie2VG","_version":1,"result":"created","_shards":{"total":2,"successful":1,"failed":0},"_seq_no":2,"_primary_term":1,"status":201}},{"_index":{"_index":"alerts-demo-0001","_id":"48T1bp0BfNVXN1ie2VG","_version":1,"result":"created","_shards":{"total":2,"successful":1,"failed":0},"_seq_no":3,"_primary_term":1,"status":201}},{"_index":{"_index":"alerts-demo-0001","_id":"48T1bp0BfNVXN1ie2VG","_version":1,"result":"created","_shards":{"total":2,"successful":1,"failed":0},"_seq_no":4,"_primary_term":1,"status":201}]}]colenan@ubuntu:~$ curl "http://192.168.56.112:9200/alerts-demo-0001/_count"
{"count":5,"_shards":{"total":2,"successful":1,"skipped":0,"failed":0}}colenan@ubuntu:~$
```

Creating the Data View in Kibana



Alerts Dashboard

- Alerts by Severity
- Top Alerting Hosts
- Alerts Over Time



Results

- Kibana and Elasticsearch services successfully connected.
- Alerts simulated and indexed correctly.
- SOC dashboard visualized trends and severity distributions.
- Index and Server Log connectors tested successfully.
- Email connector was not used, maintaining offline simulation integrity.

Future Enhancements

- Integrate **Winlogbeat** or **Filebeat** to import live system logs.
- Configure **Slack** or **Webhook** connectors for external alerting.
- Add role-based access controls within Kibana.
- Automate index creation and alert ingestion using Bash or Python scripts.

Next Steps

- Configure and test alerts in Kibana/Elastic for automated incident notification.
- Expand monitoring to include network telemetry and cloud logs.
- Add more complex SOC-style detection rules (e.g., brute force login attempts, suspicious process chains).
- Integrate additional log sources like Active Directory, Azure AD, and VPN logs for comprehensive monitoring.

Conclusion

This project successfully demonstrates a realistic SOC workflow for Windows endpoints using open-source tools by simulating, visualizing, and analyzing alert data in Kibana. Through data indexing, connectors, and custom dashboards, it models how security teams can continuously monitor processes, registry changes, file creation, PowerShell activity, Windows Defender alerts, and lateral movement—maintaining situational awareness and detecting threats without relying on external services.